

CENTRALE LYON

MOD 7.2 : Introduction aux sciences des données
Rendu BE 1

Derrière l'écran : reconnaître les utilisateurs de Copilote par le rythme et l'enchaînement de leurs actions

Élèves :
Abderrazak Khalil
Salma Ihssane
Anas Meniari

Enseignants :
Antoine Becquet

Table des matières

1 Comprendre les traces avant de construire un modèle **1**
1.1 Audit de forme et de volume 1
1.2 Analyse de la cible 2
1.3 Alignement entre apprentissage et test 2
1.4 Sparsité et décomposition des actions 3
1.5 Dynamique séquentielle et temporelle 4

2 Transformer les sessions en variables d'apprentissage **4**
2.1 Une première représentation par l'encodage séquentiel des traces 4
2.2 Une seconde représentation par le comportement global 5
2.3 Combiner les deux représentations sans écraser un signal 5

3 Faire progresser la solution par étapes **5**
3.1 Croiser trois représentations et cinq familles de modèles 5
3.2 Du modèle isolé au vote souple : un premier palier à 97,4 % 6
3.3 Itérations intermédiaires, rééquilibrage global et saut final à 98,4 % 6

4 Résultats et enseignements **7**
4.1 Synthèse de notre progression 7
4.2 Un apprentissage collectif et une démarche assumée 7
4.3 Bilan et pistes d'amélioration 8

1 Comprendre les traces avant de construire un modèle

Peut-on reconnaître une personne à sa manière d'utiliser un logiciel ? C'est la question que nous avons étudiée à partir des traces de l'ERP Copilote, développé par Infologic. Chaque session rassemble un navigateur, des actions et des repères temporels ; nous devons lui attribuer l'un des **247 utilisateurs** présents dans l'apprentissage.

Pour mieux comprendre à quoi correspondaient ces traces, nous avons aussi consulté la présentation de Copilote proposée par Infologic, notamment son « usine cliquable » (figure 1). Elle situe les usages dans leur contexte métier : réceptions et stocks, fabrication, préparation, expéditions et administratif. Cette vue nous a aidés à rapprocher les actions et les écrans des parcours de travail possibles dans l'ERP.



FIGURE 1 – Vue de l'« usine cliquable », consultée pour situer les activités couvertes par Copilote. Source : Infologic.

La préparation a pris une grande place dans notre travail. Avant de comparer les modèles, il nous fallait comprendre ce que contenaient les lignes et ce que nous pouvions réellement en tirer. Nous avons organisé cette exploration en cinq analyses, avec l'idée de faire suivre chaque observation d'une décision pour la suite. Ce lien entre comprendre les données et choisir une méthode a été au centre de notre apprentissage.

1.1 Audit de forme et de volume

En parcourant les fichiers bruts, nous constatons que chaque ligne correspond à une session, précédée de l'utilisateur et du navigateur dans le train, mais seulement du navigateur dans le test. Comme les sessions ont des **longueurs très variables**, nous avons retenu une **lecture ligne par ligne**, sans créer de colonnes vides pour les plus courtes.

	Apprentissage	Test
Nombre de sessions	3 279	324
Nombre médian d'actions	340	365
Nombre moyen d'actions	692,4	662,3
Nombre maximal d'actions	11 977	6 203

TABLE 1 – Volume des données, en comptant les actions séparément des horloges.

Quelques sessions très longues expliquent **l'écart entre moyenne et médiane**, tandis que trois sessions d'apprentissage contiennent moins de cinq actions, dont une sans action. Pour éviter de distinguer surtout les sessions par leur longueur, nous avons privilégié les **fréquences relatives** et les **transformations logarithmiques** plutôt que les seuls comptes bruts.

Fig. 1 - Distribution du nombre d'actions par session

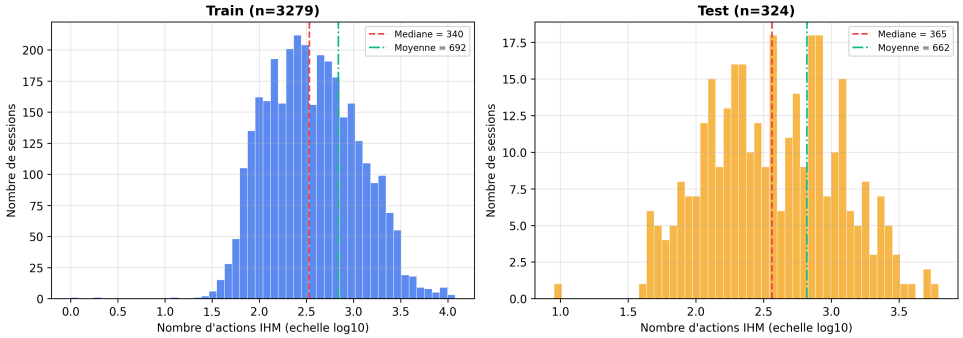


FIGURE 2 – Longueurs des sessions : une forte hétérogénéité dans les deux ensembles.

1.2 Analyse de la cible

Les utilisateurs ont entre **4 et 75 sessions**, avec une médiane de 12 ; 33 personnes n'en ont pas plus de cinq. Ce déséquilibre nous laisse peu d'exemples pour certains utilisateurs et nous a conduits à retenir le **Macro-F1**, qui donne le même poids à chacun. Le minimum de quatre sessions fixe aussi notre validation stratifiée à **quatre plis**, le maximum permettant de retrouver chaque utilisateur dans chaque pli de validation externe.

Fig. 2 - Desequilibre de la variable cible (247 utilisateurs)

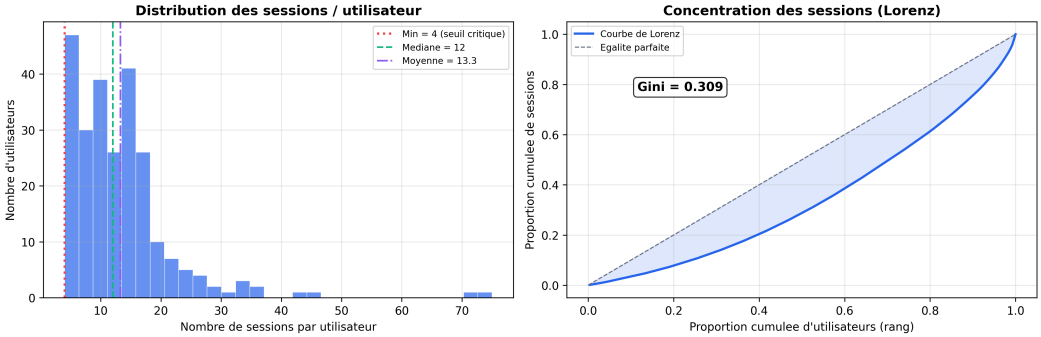


FIGURE 3 – Répartition des sessions par utilisateur et concentration des effectifs.

1.3 Alignement entre apprentissage et test

Les longueurs de sessions sont assez proches entre train et test, tout comme la répartition des navigateurs : Firefox représente respectivement 44,7 % et 45,4 % des sessions, Chrome 40,8 % et 41,0 %. Nous n'avons donc pas relevé de **décalage majeur** sur ces caractéristiques, sans pouvoir exclure un *covariate shift* sur d'autres comportements.

Nous avons aussi remarqué que les 247 utilisateurs conservaient chacun le **même navigateur** dans le train. Nous pouvions alors entraîner des **modèles distincts** par navigateur, ou garder un apprentissage commun puis filtrer les prédictions. La première approche aurait demandé davantage d'entraînements et de réglages, sans garantie de gain en l'absence de **benchmark**. Nous avons donc conservé le navigateur comme variable indicatrice et retenu un **post-traitement** écartant les identités associées à un autre navigateur, avec le risque d'exclure la bonne personne si elle en change dans le test.

Fig. 3 - Analyse du navigateur (contexte & fidelite)

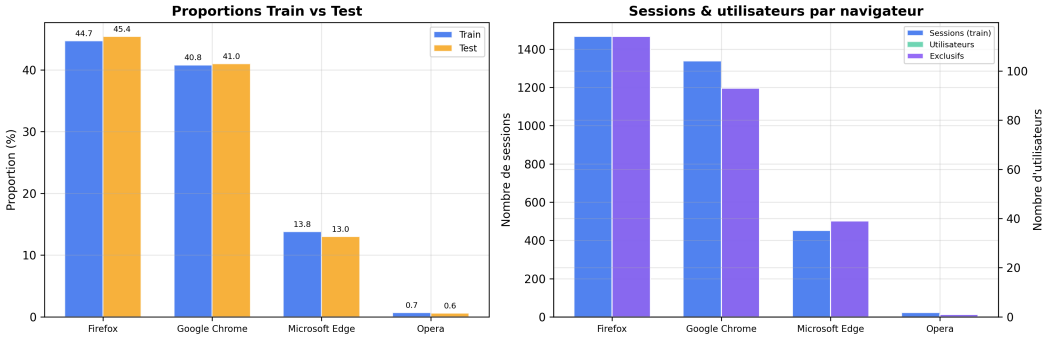


FIGURE 4 – Comparaison train/test et fidélité des utilisateurs à leur navigateur.

1.4 Sparsité et décomposition des actions

Parmi les **7 086 tokens distincts** du train, 28 % n'apparaissent qu'une fois et 56,5 % au plus cinq fois ; le test contient aussi **206 tokens nouveaux**, soit 7,3 % de son vocabulaire distinct. Encoder uniquement les événements complets nous aurait donc laissés avec beaucoup de motifs rares et des événements inconnus.

L'inspection des traces révèle toutefois une structure commune :

```
Action(Controleur)<Configuration>$Domaine$1
```

Les composants après l'action sont optionnels, mais isolent le geste, l'écran, sa configuration et le domaine. Nous avons donc conservé à la fois le **token complet** et ses **composants**, afin que des événements différents puissent partager une information déjà connue.

Niveau	Vocabulaire du train	Éléments nouveaux du test
Action de base	35	0
Contrôleur	415	8
Configuration	219	2
Domaine	76	0

TABLE 2 – La décomposition réduit la difficulté liée aux événements inconnus.

Nous avons également adapté le **parseur** à quelques exceptions : un marqueur `t5` collé à une action par une virgule manquante et des parenthèses imbriquées dans des traces Java. Sans ce traitement, le découpage aurait créé de fausses catégories ou perdu des repères temporels.

Fig. 4 - Sparsite lexicale & gain de la decomposition

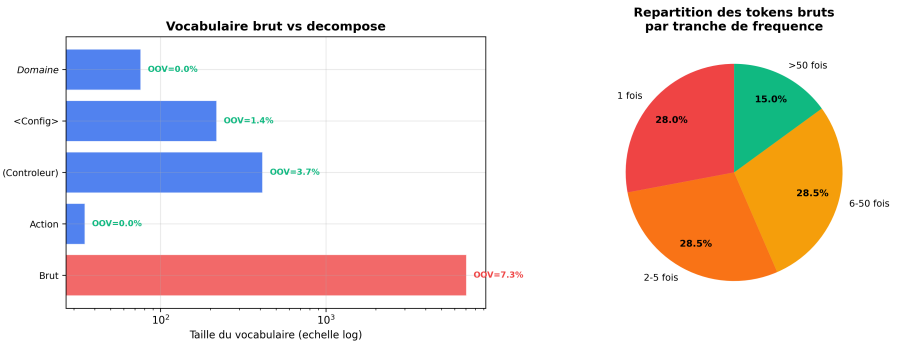


FIGURE 5 – Rareté des tokens bruts et intérêt de leur décomposition.

1.5 Dynamique séquentielle et temporelle

L'exécution d'un bouton représente 22,49 % des actions, la saisie 11,70 % et la sélection d'un écran 9,11 %. Ces gestes courants nous renseignent moins par leur présence que par leurs **proportions** et leurs **enchaînements**. Les **18,08 % de répétitions immédiates** d'une même action de base nous ont notamment amenés à conserver les **séries répétées**.

Les marqueurs `tX`, insérés toutes les cinq secondes, permettent d'estimer une durée médiane de session de 395 secondes et d'analyser le **rythme d'activité**. En inspectant ces jalons, nous avons toutefois repéré des doublons temporels, comme un marqueur `t10` réapparaissant une seconde fois plus loin dans la même trace. Loin d'être un simple bruit, ce chevauchement traduit probablement **l'ouverture de plusieurs fenêtres en parallèle** par l'utilisateur. Ces repères ne donnant pas l'heure exacte de chaque clic, nous avons choisi de **mesurer l'activité par tranches de cinq secondes** tout en protégeant nos calculs de débit contre ces intervalles nuls.

Nous avons ainsi retenu les **courts motifs**, les **séries répétées** et les **statistiques temporelles** pour décrire, au-delà des écrans visités, la manière dont chaque utilisateur travaille.

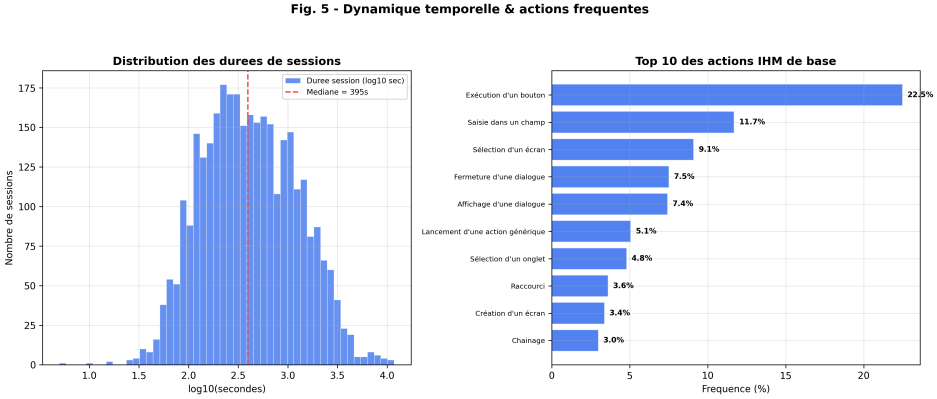


FIGURE 6 – Durées des sessions et principales actions observées.

2 Transformer les sessions en variables d'apprentissage

2.1 Une première représentation par l'encodage séquentiel des traces

Dans un premier temps, nous avons construit un premier jeu de données dédié uniquement à la transformation et au codage de la séquence d'événements, en assimilant chaque session à un document. Comme un simple *Bag of Words* restait trop sensible à la longueur des traces et aux répétitions, nous avons structuré cet encodage autour de cinq briques complémentaires :

- **Les unigrammes TF-IDF (fréquence sous-linéaire et norme L2)** : appliqués aux tokens complets comme à leurs sous-composants, ils capturent le vocabulaire de la session tout en atténuant le poids des actions omniprésentes.
- **Les bigrammes et trigrammes** : ils dépassent la simple présence d'un écran pour conserver les courts enchaînements de gestes.
- **Les probabilités de transition de Markov** : calculées entre les dix actions les plus fréquentes (matrice de 100 valeurs), elles modélisent la dynamique locale du flux d'un clic à l'autre.
- **Le codage par longueur de séries (*Run-Length Encoding*)** : directement inspiré des 18 % de répétitions immédiates observés lors de l'exploration, il résume les rafales d'actions identiques consécutives.

— **L'ancrage de position** : il isole les tout premiers et tout derniers événements de la trace pour capter les rituels d'ouverture et de fermeture propres à chaque opérateur.

Pour contenir la dimension de cet espace sans perdre de signal, nous avons appliqué des seuils de présence minimale et des plafonds de vocabulaire, tout en veillant à ce que le découpage préserve chaque nom d'action contenant des espaces comme une unité intacte.

2.2 Une seconde représentation par le comportement global

En parallèle de ce codage séquentiel, nous avons élaboré un second jeu de données regroupant **246 descripteurs comportementaux** qui résument la physionomie générale de la session plutôt que le détail des clics. Ce tableau synthétise le volume et la durée globale de l'activité, la diversité et l'entropie des écrans ou domaines consultés, ainsi que des indicateurs de rythme comme la vitesse d'exécution et les temps de pause calculés à partir des marqueurs temporels. Cette seconde approche offre ainsi une lecture beaucoup plus agrégée et interprétable de la manière d'utiliser Copilote, décrivant les mêmes sessions à une échelle complémentaire de la représentation lexicale.

2.3 Combiner les deux représentations sans écraser un signal

Réunir ces deux approches dans un troisième jeu de données **hybride** nous paraissait naturel pour associer la précision des motifs séquentiels à la vue d'ensemble comportementale. Pourtant, notre première concaténation directe a donné de moins bons résultats que la représentation séquentielle utilisée seule.

Cet échec initial a été particulièrement instructif : ajouter des variables pertinentes séparément ne garantit pas qu'elles fonctionneront bien ensemble si leurs amplitudes respectives entrent en conflit. En pratique, les valeurs brutes des descripteurs comportementaux prenaient le dessus numériquement et écrasaient le signal fin des vecteurs TF-IDF déjà normalisés.

Pour corriger ce déséquilibre, nous avons procédé en trois étapes sur le bloc comportemental avant la fusion :

1. une **standardisation** de chaque variable sur l'apprentissage ;
2. une **normalisation L2 par session** pour aligner sa géométrie sur celle du bloc TF-IDF ;
3. l'application d'un **coefficient de réduction de 0,25**, issu de nos premiers réglages, afin de tempérer son influence globale.

Ce rééquilibrage a immédiatement inversé la tendance : les statistiques globales viennent désormais compléter les motifs lexicaux pour lever les ambiguïtés sans jamais dominer la décision. Plus qu'un simple réglage technique, cette étape nous a appris qu'il fallait d'abord rendre deux sources d'information comparables avant de chercher à les combiner.

3 Faire progresser la solution par étapes

3.1 Croiser trois représentations et cinq familles de modèles

Pour reconnaître à la fois les tâches réalisées et la manière de les enchaîner, nous avons d'abord confronté **cinq familles de modèles** à nos trois jeux de données : régression logistique, SVM linéaire, MLP, arbre de décision et XGBoost. Tandis que les modèles linéaires exploitaient directement la matrice TF-IDF creuse, une réduction par SVD calculée dans chaque pli permettait aux arbres et au MLP de travailler sur un espace dense sans perte de généralisation.

Ce premier croisement a immédiatement orienté la suite de notre travail :

Modèle	Séquence (TF-IDF)	Comportement	Hybride initial
Régression logistique	92,36	46,93	86,76
SVM linéaire	**96,20**	65,28	87,54
MLP	87,05	77,67	85,54
Arbre de décision	19,28	65,61	62,15
XGBoost	71,70	83,01	**85,62**

TABLE 3 – Macro-F1 (%) après masque navigateur dans le benchmark exploratoire des cinq modèles.

— **La complémentarité des regards** : le SVM linéaire excelle dans la lecture des motifs séquentiels fins (96,20 %), là où XGBoost tire le meilleur parti des descripteurs comportementaux (83,01 %).
— **La preuve par l'hybride** : sur la concaténation brute, le SVM chutait à 87,54 % à cause du conflit d'échelle entre les blocs. Une fois appliquées la normalisation L2 et la pondération à 0,25, le **SVM hybride a rebondi à 96,38 %**, prouvant que les statistiques globales apportaient bien un signal supplémentaire.

3.2 Du modèle isolé au vote souple : un premier palier à 97,4 %

Pour évaluer nos idées sans favoriser les utilisateurs omniprésents, nous avons verrouillé notre protocole autour du **Macro-F1** sur une validation stratifiée à quatre plis. Surtout, nous avons traqué la moindre fuite d'information (*data leakage*) en réapprenant chaque transformation strictement dans son pli d'entraînement et en adoptant une **validation croisée imbriquée** (*nested CV*) : les boucles internes règlent les hyperparamètres et les poids du vote, tandis que la boucle externe juge la méthode à l'aveugle.

Plutôt que de chercher un unique modèle champion, nous avons fait dialoguer nos cinq familles au sein d'un **vote souple pondéré** (*soft voting*), chacune associée à sa représentation de prédilection :

— les marges du SVM sont converties en probabilités via une fonction *softmax* calibrée par température afin de s'équilibrer avec les autres classifieurs ;
— le **masque navigateur** vient clore la décision en annulant les probabilités des identités incompatibles avec le navigateur de la session, ce qui fait bondir le SVM lexical de 94,17 % à 96,20 % en validation locale.

Dès cette première soumission sur Kaggle, notre ensemble classique a atteint un score public de 97,4 %. Ce résultat validait d'emblée toute notre chaîne de préparation des données.

3.3 Itérations intermédiaires, rééquilibrage global et saut final à 98,4 %

Une fois ce socle posé, nous avons multiplié les explorations intermédiaires (passant par 97,2 %, 97,9 % puis 98,0 %) en ajoutant un classifieur **Ridge** sur l'hybride, des similarités cosinus avec le profil moyen de chaque utilisateur et un focus accru sur les cinq premières et dernières actions.

Une démarche en particulier a caractérisé cette phase : en observant que notre ensemble ne prédisait que 224 identités distinctes sur les 324 sessions de test, nous avons mis en place un **rééquilibrage global inspiré de l'algorithme de Sinkhorn**. Au lieu de trancher chaque session isolément, cette approche ajuste la matrice des scores sur l'ensemble du test pour éviter que quelques profils fréquents ne cannibalisent les décisions, redistribuant les prédictions vers 229 utilisateurs tout en respectant le masque navigateur.

Pour franchir un dernier cap, nous avons finalement changé de paradigme en construisant une

architecture de *deep learning* séquentiel sur mesure : **ConvBiGRUAttention**. Là où nos modèles précédents comptaient des motifs, ce réseau lit véritablement la session comme une histoire chronologique :

- il associe chaque événement à sa position dans la trace et à son rythme temporel estimé ;
- ses couches convolutives capables de capter les gestes locaux nourrissent une GRU bidirectionnelle qui suit la dynamique dans les deux sens du temps ;
- un mécanisme d'attention vient enfin pointer les instants les plus signants de la session pour identifier l'opérateur.

Travaillant sans GPU dédié, avec des temps de calcul importants sur CPU, nous avons dû brider la fenêtre d'entrée du réseau à **126 événements** et restreindre la recherche d'hyperparamètres. Malgré cette contrainte matérielle forte, cette approche neuronale a hissé notre ultime soumission à **98,4 % sur le leaderboard public**, clôturant nos efforts sur notre meilleur score.

4 Résultats et enseignements

4.1 Synthèse de notre progression

Le tableau suivant retrace la trajectoire de nos soumissions publiques, du premier jet jusqu'à l'architecture finale :

Étape du projet	Score public Kaggle (%)
Première soumission : Socle feature engineering + Vote souple des 5 modèles	**97,4**
Soumissions intermédiaires : Ridge, profils cosinus et rééquilibrage de Sinkhorn	97,2 à 98,0
Soumission finale : Approche séquentielle profonde (**ConvBiGRUAttention**)	**98,4**

TABLE 4 – Progression des performances publiques au fil de nos hypothèses.

Ce parcours montre deux choses : d'abord, qu'une ingénierie des variables réfléchie et ancrée dans le métier permet d'atteindre immédiatement un niveau très élevé (97,4 %) avec des modèles classiques bien combinés ; ensuite, que le rééquilibrage global et la modélisation neuronale de la séquence complète apportent la finesse nécessaire pour départager les sessions ambiguës et gagner le point décisif jusqu'à 98,4 %.

4.2 Un apprentissage collectif et une démarche assumée

La réussite de ce projet tient d'abord à la **complémentarité de notre groupe** : associer un membre déjà expérimenté en *machine learning* à des profils au regard neuf, curieux et critique a créé une vraie émulation. Les questions de fond des uns obligeaient à vulgariser et à justifier chaque choix sans jamais appliquer de recette toute faite, permettant à chacun de consolider ses acquis et de maîtriser toute la chaîne.

Cette dynamique a été décuplée par le format en **compétition Kaggle** : voir le classement évoluer nous poussait sans cesse à remettre nos idées sur l'établi plutôt que de nous contenter du premier score à 97,4 %. Dans cette course, les modèles de langage et agents de codage nous ont servi d'**assistants d'implémentation et de débogage** pour accélérer l'écriture du code lourd. En revanche, toutes les décisions scientifiques (du diagnostic du double `t10` et du rééquilibrage de Sinkhorn jusqu'à l'architecture **ConvBiGRUAttention**) sont nées de nos débats d'équipe, chaque script étant relu, testé et compris avant validation.

4.3 Bilan et pistes d'amélioration

Ce projet nous laisse avant tout un réflexe d'ingénieur : toujours partir de la réalité physique et métier de la donnée avant d'empiler des algorithmes.

Nous nous sommes arrêtés à 98,4 % pleinement satisfaits du chemin parcouru avec nos moyens de calcul. Si nous devions poursuivre l'expérience sur un serveur équipé de GPU, la suite logique serait de libérer notre architecture **ConvBiGRUAttention** en portant sa capacité d'entrée de **126 à 512 tokens ou davantage**. Le réseau pourrait alors absorber l'intégralité des sessions longues sans troncature et exprimer tout son potentiel face aux approches statistiques.

Code, figures et tableaux : `https://github.com/Abderrazakkhalil/copilote-trace-mining`.